

Spring Security Roadmap (Practical + Interview Ready)

◆ Phase 0: Prerequisites (Don't skip)

Before Spring Security, be clear on:

- Spring Boot basics (controllers, services, filters)
 - REST APIs
 - HTTP concepts (headers, cookies, sessions, status codes)
-

◆ Phase 1: Core Fundamentals (Must Know)

👉 Goal: Understand *what is happening under the hood*

Topics:

- What is Spring Security?
- Authentication vs Authorization
- Filter Chain (VERY IMPORTANT)
- Security Context
- Basic flow of login

Key Concept:

👉 Every request passes through a **filter chain**

🔑 Learn This Properly:

- UsernamePasswordAuthenticationToken
- AuthenticationManager
- SecurityContextHolder

👉 Don't memorize—understand flow.

◆ Phase 2: Basic Implementation (Hands-on)

👉 Goal: Secure a simple REST API

Build:

- Spring Boot project
- Add Spring Security dependency
- Create 2 endpoints:

- /public → no auth
- /private → secured

Learn:

- Default login behavior
 - In-memory authentication
-

 Code Concepts:

- SecurityFilterChain
 - UserDetailsService
 - Password encoding (BCryptPasswordEncoder)
-

 Phase 3: Role-Based Authorization

 Goal: Control access

Learn:

- Roles (ROLE_USER, ROLE_ADMIN)
- Endpoint security

Example:

- /admin → only ADMIN
 - /user → USER + ADMIN
-

 Important:

- .hasRole()
- .hasAuthority()

 Interview favorite topic 

 Phase 4: JWT Authentication (🔥 MOST IMPORTANT)

 This is what most companies expect

Learn JWT Flow:

1. User logs in

2. Server generates token
 3. Client sends token in header
 4. Server validates token
-

Key Concepts:

- Stateless authentication
 - No session
 - Token validation
-

Implement:

- Login API → returns JWT
 - Create JWT Utility class
 - Custom filter for token validation
-

Important Classes:

- OncePerRequestFilter
 - JwtUtil
 - UserDetailsService
-

◆ Phase 5: Exception Handling & Real-world Setup

 Make your project production-like

Learn:

- Custom authentication entry point
 - Exception handling
 - Unauthorized vs Forbidden (401 vs 403)
-

◆ Phase 6: Advanced (Only if time permits)

 Do NOT rush here unless basics are solid

Topics:

- OAuth2 (Google login, etc.)
- Method-level security (@PreAuthorize)

- Custom filters
- CSRF
- CORS

◆ Phase 7: Build One Solid Project (VERY IMPORTANT)

👉 This is your interview weapon

💡 Project Idea:

Secure E-commerce Backend

Features:

- User registration & login
- JWT authentication
- Role-based access:
 - USER → view products
 - ADMIN → add/delete products
- Exception handling
- Password encryption

👉 If you can explain THIS project, you're safe in interviews.

📅 Suggested Timeline (Realistic)

Day Focus

Day 1 Fundamentals

Day 2 Basic security + roles

Day 3 JWT implementation

Day 4 Polish + exceptions

Day 5 Build project

🎯 Interview Preparation Strategy

Focus on explaining:

1. Flow (VERY IMPORTANT)

“Request → Filter → Authentication → SecurityContext → Controller”

2. JWT Flow

“Client sends token → filter validates → user set in context”

3. Key Difference

- Authentication = Who are you?
 - Authorization = What can you access?
-

❌ Common Mistakes to Avoid

- Jumping directly to JWT without basics
 - Memorizing configs without understanding flow
 - Adding Spring Security in resume without a project
-

🔥 Final Outcome (What you should be able to say)

“I have implemented Spring Security with JWT-based authentication, role-based authorization, and secured REST APIs using filter chains and custom validation logic.”

💬 If you want next step:

I can:

- Build a **complete production-level GitHub project (with JWT + roles + best practices)**
- Give you **interview Q&A with answers you can actually speak confidently**
- Or create a **cheat sheet for quick revision before interviews**

Just tell me 👍

🎯 Goal

- /public → accessible without login
- /private → requires authentication

We'll use **basic auth (simple & interview-friendly)**.

📁 1. Project Setup

Dependencies (Maven)

```
<!-- pom.xml -->
<dependencies>

    <!-- Spring Boot Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

</dependencies>
```

2. Project Structure

```
src/main/java/com/example/demo
├── DemoApplication.java
├── config/SecurityConfig.java
└── controller/TestController.java
```

3. Main Class

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

4. Security Configuration

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
```

```
@Configuration
```

```
public class SecurityConfig {
```

```
    @Bean
```

```
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable()) // disable for REST APIs
```

```
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/public").permitAll()
```

```
                .requestMatchers("/private").authenticated()
```

```
                .anyRequest().authenticated()
```

```
            )
```

```
            .httpBasic(); // enable basic auth
```

```
        return http.build();
```

```
    }
```

```
// In-memory user (for testing)
```

```
@Bean
```

```
public UserDetailsService userDetailsService() {
```

```
    return new InMemoryUserDetailsManager(
```

```
        User.withUsername("monika")
```

```
            .password("{noop}123") // {noop} = no encoding (for demo only)
```

```
            .roles("USER")
```

```
            .build()
```

```
    );
```

```
    }
```

```
}
```

5. Controller

```
package com.example.demo.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class TestController {
```

```
    @GetMapping("/public")
```

```
public String publicApi() {  
    return "This is PUBLIC endpoint (no auth required)";  
}  
  
@GetMapping("/private")  
public String privateApi() {  
    return "This is PRIVATE endpoint (authentication required)";  
}  
}
```

6. Run the Application

Start your app:

```
mvn spring-boot:run
```

7. Testing APIs

Public API (No Auth)

GET <http://localhost:8080/public>

 Response:

This is PUBLIC endpoint (no auth required)

Private API (Requires Auth)

GET <http://localhost:8080/private>

 Without auth:

401 Unauthorized 

With Basic Auth

Use:

- Username: monika
- Password: 123

Curl:

```
curl -u monika:123 http://localhost:8080/private
```

 Response:

This is PRIVATE endpoint (authentication required)

What happens internally (connect theory)

1. Request hits **Security Filter Chain**
2. /public → allowed directly
3. /private → triggers authentication
4. BasicAuth sends credentials
5. Spring creates **Authentication object**
6. Validates using UserDetailsService
7. Stores in **SecurityContextHolder**

Key Concepts Covered

- ✓ Filter Chain
- ✓ Authentication
- ✓ SecurityContextHolder
- ✓ In-memory users
- ✓ Basic Auth

Important Notes

- {noop} → only for demo (no password encoding)
- In real apps → use BCryptPasswordEncoder
- Basic Auth → sends credentials every request (not ideal for production)

Next Step (Recommended)

Now that you understand this, next level is:

-  Add **JWT authentication (stateless)**
-  OR connect with **database (UserDetailsService + JPA)**

If you want, I can convert this into:

- ✓ Full GitHub project (production-ready)
- ✓ JWT version of the same API (very important for interviews)

